



**COMSATS University Islamabad
Abbottabad, Pakistan**

UART-SIM

**A Software Simulation of a UART Peripheral for Register-Level Design and
Verification**

By

Ali Hassan

25ins-04020

Inam Ullah Khan

25ins-04008

Abdullah Zeb

25ins-04001

Final Project Report

C/C++ for Hardware Engineers

Table of Contents

1. Introduction.....	1
1.1 Brief.....	1
1.2 Relevance to Course Modules.....	1
1.3 Project Background	1
1.4 Literature Review	1
1.5 Analysis from Literature Review	1
1.6 Methodology and Software Lifecycle	2
1.6.1 Rationale behind the Selected Methodology	2
1.6.2 Rationale behind the Selected Software Lifecycle	2
2. Problem Definition.....	3
2.1 Problem Statement	3
2.2 Deliverables and Development Requirements	3
2.2.1 Deliverables	3
2.2.2 Development Requirements	3
2.3 Existing Approaches	3
3. Functional Requirements	4
3.1 Use Case Diagram.....	4
3.2 Detailed Use Cases.....	4
3.3 Functional Requirements.....	5
3.4 Non-Functional Requirements	5
4. System Architecture.....	6
4.1 Data Representation	6
4.2 Process Flow	6
4.3 Design Models.....	6
4.4 Data Flow Diagram	7
5. Implementation	8
5.1 Algorithms.....	8
5.1.1 Frame Encode/Decode Algorithm	8
5.1.2 Circular Buffer Management Algorithm	8
5.2 Public Interface (Class API).....	8

5.3 Console Interface.....	9
5.3.1 Register and Status Display	9
5.3.2 Framed Bit Sequence Display	9
5.3.3 Event Log Output	9
6. Conclusion and Future Work.....	10
6.1 Conclusion.....	10
6.2 Future Work	10
7. Division of Work	11
7.1 Topic Coverage Mapping.....	11
8. References.....	12

1. Introduction

1.1 Brief

UARTSim is a software simulation of a Universal Asynchronous Receiver/Transmitter (UART) built in C++ as the final project for the C/C++ for Hardware Engineers module. It simulates memory-mapped registers, UART data transmission, transmit and receive circular buffers, a peripheral hierarchy using polymorphism, and a callback system using lambda functions. The project uses two connected UART instances to show how data is sent from one UART and received by the other.

1.2 Relevance to Course Modules

UART peripherals are among the most common interfaces in embedded and hardware systems, used in debug consoles, sensor links, and inter-chip communication. Modeling one in software directly exercises register-level programming, bit manipulation, data structure design, object-oriented modeling, and event-driven callback patterns, which together map onto every core topic area covered in this module.

1.3 Project Background

The project was undertaken in a group of three as part of the final assessment for the C/C++ for Hardware Engineers module, which requires a software-only simulation tool relevant to chip design, verification, or embedded systems. UART was selected because it is small enough in scope to implement and verify within the project timeline, while still being representative of the register-bank, protocol-framing, and interrupt-handling concerns found in larger peripheral models.

1.4 Literature Review

Background reading for this project drew on publicly documented UART peripheral designs, including the classic 8250/16550-family register layout and the ARM PL011 UART technical reference manual, both of which describe a DATA/STATUS/CONTROL-style register bank similar to the one modeled in UARTSim. General serial-framing conventions (start bit, configurable data width, optional parity, stop bit) follow the long-standing RS-232 asynchronous serial convention. On the software side, the project's circular buffer and callback-based event design follow patterns commonly described in embedded systems and real-time systems textbooks for modeling FIFO-based peripheral queues and interrupt service routine behavior in a polling or tick-driven context.

1.5 Analysis from Literature Review

Most introductory UART examples in embedded systems teaching material isolate a single concept at a time, for instance a register-access demo, or a standalone circular buffer, without combining them into one coherent peripheral model. UARTSim was designed to close that gap

for this project by integrating register-level access, bit-accurate framing, buffered queuing, and interrupt-style callbacks into a single tool, and by connecting two instances together so that the combined behavior can be observed end-to-end rather than tested in isolation.

1.6 Methodology and Software Lifecycle

Development followed a lightweight, iterative approach: each of the three core subsystems (register/framing engine, buffer/bus layer, and event/callback layer with the test harness) was assigned to one group member and built incrementally, then integrated once the individual pieces reached a working state.

1.6.1 Rationale behind the Selected Methodology

An iterative, feature-driven approach was chosen because the project's scope was fixed by the assignment brief and divided naturally along the six required topic areas. Instead of using a strict formal process, each team member developed and tested their own part separately based on the agreed interface (the `UartRegisters` struct, the `CircularBuffer` template, and the callback signatures). This helped reduce integration problems because the team had limited time.

1.6.2 Rationale behind the Selected Software Lifecycle

An incremental lifecycle model was used in preference to a strict waterfall model. Register and framing logic were implemented and tested first, since the buffer and callback layers depend on them; the circular buffer and bus layer followed; the event system and integration test harness were added last, once the underlying components were stable. This ordering matched the natural dependency chain between the three members' responsibilities.

2. Problem Definition

2.1 Problem Statement

Serial communication peripherals such as the UART must be modeled and verified at the register and protocol level in software before being committed to silicon or firmware. UARTSim addresses this by providing a software simulation of a UART peripheral consisting of memory-mapped control, status, and data registers, a bit-level framing engine that constructs and parses serial frames, and transmit/receive circular buffers that model realistic hardware queuing behavior, including overflow conditions, together with an interrupt-style callback system for byte-received, buffer-overflow, and framing-error events.

2.2 Deliverables and Development Requirements

2.2.1 Deliverables

- Complete C++ source code for the UARTSim tool (register bank, framing engine, circular buffers, peripheral/bus hierarchy, event system, test harness).
- This project report (10-12 pages), including the academic integrity check.
- A 20-30 minute oral viva in which all group members explain their component and design choices.
- A working console demonstration transmitting a message between two connected UART instances, including fault-injection scenarios.

2.2.2 Development Requirements

- Implementation strictly in C++, compiled with a standard-conformant compiler (e.g. g++ or clang++).
- No physical hardware or external toolchain dependency; the tool must run entirely in software.
- Coverage of at least four of the six required topic areas (UARTSim covers all six).
- Modular source layout allowing each member's component to be developed and tested independently.

2.3 Existing Approaches

In industry, UARTs are usually tested using HDL testbenches during chip design or by using real hardware with a logic analyzer after the chip or development board is available. However, these methods require special software or hardware, which was not available for this project. UARTSim provides a simple software-only solution that simulates UART registers and communication behavior without needing an HDL simulator or physical UART hardware.

3. Functional Requirements

3.1 Use Case Diagram

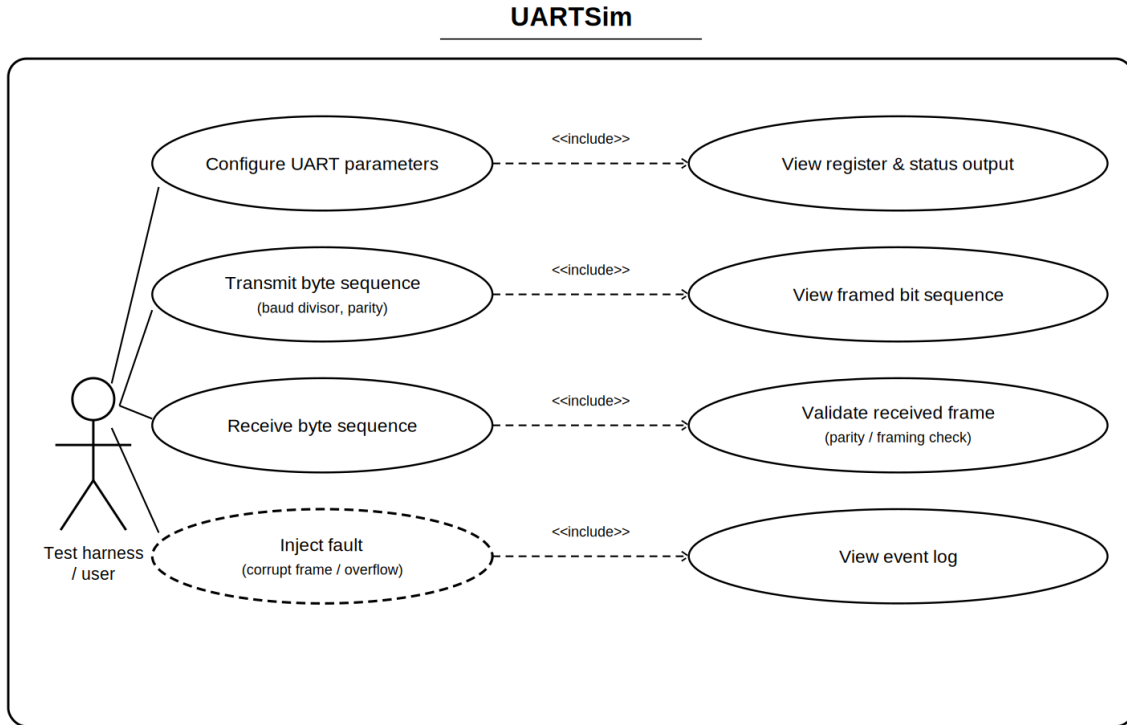


Figure 1: UARTSim use case diagram

3.2 Detailed Use Cases

Use Case	Description
Configure UART parameters	The user sets baud divisor and parity enable/disable via command-line configuration before a simulation run.
Transmit byte sequence	The user supplies a message or byte sequence to be encoded into frames and transmitted from one Uart instance.
Receive byte sequence	The connected Uart instance decodes incoming frames from its RX buffer and reconstructs the original bytes.
Inject fault	The user issues a fault-injection command to corrupt a frame or force a buffer overflow, to test error detection.
View register / status output	The user observes register and status-flag states (TX_BUSY, RX_READY, FRAMING_ERROR, PARITY_ERROR, BUFFER_OVERFLOW) after each cycle.
View event log	The user observes the callback-driven event log generated on byte-received, buffer-overflow, and framing-error conditions.

3.3 Functional Requirements

- FR1: The system shall encode a byte into a start+data+parity+stop bit frame using bit shifting and masking.
- FR2: The system shall decode a received frame back into a byte and independently recompute parity.
- FR3: The system shall raise a PARITY_ERROR or FRAMING_ERROR status flag when frame validation fails.
- FR4: The system shall queue transmitted and received bytes in fixed-size circular buffers and raise BUFFER_OVERFLOW when capacity is exceeded.
- FR5: The system shall dispatch tick() polymorphically across all peripherals attached to a Bus.
- FR6: The system shall invoke user-attached lambda callbacks on byte-received, buffer-overflow, and framing-error events.

3.4 Non-Functional Requirements

- Portability: the tool must build and run with a standard C++ compiler without platform-specific dependencies.
- No hardware dependency: the entire simulation must run in software on a standard development machine.
- Maintainability: subsystems (registers, buffers, bus, callbacks) are decoupled behind clear interfaces so each can be modified independently.
- Determinism: given the same configuration and input sequence, the simulation must produce the same register, buffer, and event-log output on every run.

4. System Architecture

4.1 Data Representation

Each Uart instance owns a UartRegisters structure representing the DATA, STATUS, CONTROL, and BAUDDIV registers as they would appear in a memory-mapped peripheral. Access is performed through bit masking and shifting rather than direct field assignment, so that status flags such as TX_BUSY, RX_READY, FRAMING_ERROR, PARITY_ERROR, and BUFFER_OVERFLOW behave as they would on real hardware, and setting one flag cannot accidentally corrupt an adjacent one.

4.2 Process Flow

The simulation advances in discrete ticks. On each tick, the Bus calls tick() on every attached peripheral through a base-class pointer. Each Uart instance's tick() drives its framing engine: encoding queued bytes from the TX buffer into frames, transferring the frame stream to the connected peripheral's RX path, and decoding incoming frames into the RX buffer while updating status flags and firing any attached event callbacks.

4.3 Design Models

The class design centers on a Peripheral base class exposing a virtual tick() method. Uart derives from Peripheral and owns a UartRegisters instance and a pair of CircularBuffer instances for TX and RX queuing; an optional Spi class demonstrates that the same interface can support a second, structurally different peripheral. A Bus class holds a collection of Peripheral pointers and dispatches tick() polymorphically across all attached devices.

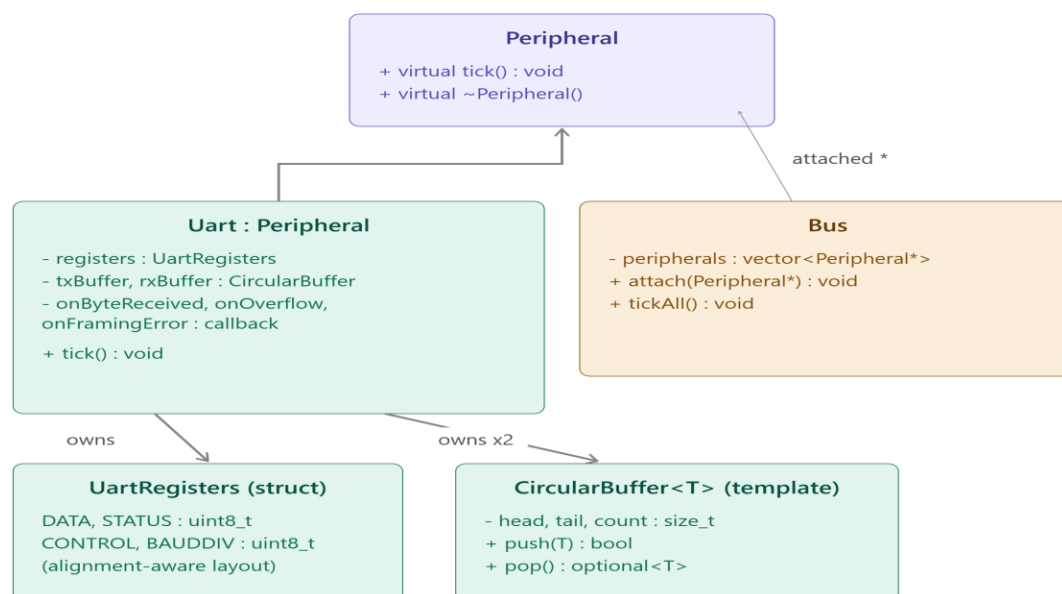


Figure 2: UARTSim class design model

4.4 Data Flow Diagram

The diagram below traces a byte from configuration input through encoding, buffered transmission, decoding, and final status/event output.

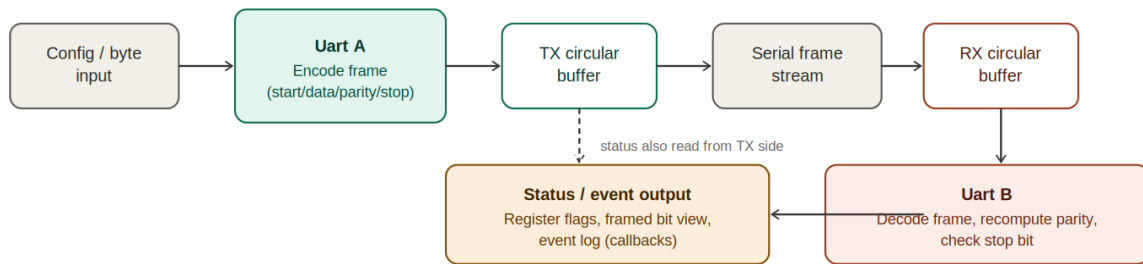


Figure 3: UARTSim data flow

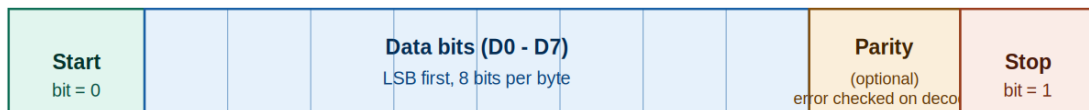
5. Implementation

5.1 Algorithms

5.1.1 Frame Encode/Decode Algorithm

Encoding walks the eight data bits of a byte from least significant to most significant, shifting each bit into its frame position between the fixed start bit (0) and stop bit (1), and computing a running parity value alongside it when parity is enabled. Decoding reverses this process: it walks the same bit positions, reconstructs the byte, and independently recomputes the expected parity bit. A mismatch between the recomputed and received parity raises `PARITY_ERROR`; a missing or malformed stop bit raises `FRAMING_ERROR`, both surfaced through the `STATUS` register.

UART frame structure



Encoding shifts and masks each data bit into the frame; decoding walks the same positions, recomputes parity, and checks the stop bit to flag `PARITY_ERROR` / `FRAMING_ERROR`.

Figure 4: UART frame structure (start, data, parity, stop)

5.1.2 Circular Buffer Management Algorithm

Each circular buffer tracks head and tail indices modulo its fixed capacity. A push operation checks whether the buffer is full before writing; if full, it raises `BUFFER_OVERFLOW` rather than overwriting unread data. A pop operation checks whether the buffer is empty before reading. The same templated implementation is reused for both TX and RX queues, and a heap-allocated variant demonstrates manual memory management alongside the stack-allocated form.

5.2 Public Interface (Class API)

The core public interface exposed by `UARTSim` consists of: `Peripheral::tick()`, overridden by `Uart::tick()` and `Bus::tickAll()`; `Bus::attach(Peripheral*)` for connecting devices to the simulation loop; `CircularBuffer::push()/pop()` for buffer access; and callback-registration methods on `Uart` for attaching lambda handlers to byte-received, buffer-overflow, and framing-error events. This interface is what the test harness in `main.cpp` uses to configure, connect, and drive the two simulated UART instances.

5.3 Console Interface

5.3.1 Register and Status Display

After each simulation cycle, the console prints the current state of the DATA, STATUS, CONTROL, and BAUDDIV registers, along with the individual status flags (TX_BUSY, RX_READY, FRAMING_ERROR, PARITY_ERROR, BUFFER_OVERFLOW) for both connected UART instances.

5.3.2 Framed Bit Sequence Display

For each byte transmitted, the console prints the constructed bit sequence (start, data, parity, stop), allowing the exact frame layout to be inspected alongside the resulting register state.

5.3.3 Event Log Output

The event log, populated through the lambda-based callback hooks, prints a line for every byte-received, buffer-overflow, or framing-error event as it occurs during the simulation, mirroring how embedded firmware would log interrupt activity.

6. Conclusion and Future Work

6.1 Conclusion

UARTSim demonstrates a complete, software-only simulation of a UART peripheral, covering all six required topic areas of the C/C++ for Hardware Engineers module. By modeling memory-mapped registers, bit-accurate framing, circular-buffer queuing, polymorphic peripheral dispatch, and lambda-based interrupt callbacks within a single cohesive tool, the project provides a scaled-down but faithful analogue of the register-level and protocol-level modeling that underpins real hardware verification workflows. Connecting two UART instances end-to-end further validates that the simulated register, buffer, and framing logic interoperate correctly across a full transmit-to-receive communication cycle.

6.2 Future Work

- Extend the Peripheral hierarchy to include a fully implemented Spi class alongside Uart.
- Add configurable baud-rate timing so the simulation models transmission delay rather than an instantaneous tick-based transfer.
- Add a simple graphical front end to visualize register and buffer state changes in real time.
- Expand fault-injection coverage to include noise-induced multi-bit corruption and clock-drift scenarios.

7. Division of Work

Member	Responsibilities
Ali Hassan	UART core, registers, and bit-level framing: memory-mapped UartRegisters struct, bit-level frame encoder/decoder with parity and framing-error detection, and the Uart class together with the Peripheral base class.
Inamullah Khan	Circular buffers, bus, and memory management: templated circular buffer for TX/RX queues with overflow detection, the Bus class managing multiple attached peripherals polymorphically, and the heap-allocated buffer variant demonstrating manual memory management.
Abdullah Zeb	Event system, callbacks, and interface: lambda-based event-hook system for interrupt-style callbacks, and the test harness/main.cpp connecting two UART instances, driving the simulation loop, and injecting faults. Also coordinated report compilation.

7.1 Topic Coverage Mapping

Topic Area	Implementation in UARTSim
C Fundamentals & Memory	Tick-based simulation loop control flow; stack vs. heap allocation of buffer storage; core data types for registers and byte streams.
Pointers, Arrays & Structs	UartRegisters memory-mapped struct; pointer-based connection between two Uart instances; alignment-aware register layout.
Low-Level & Bit Operations	Frame construction/parsing via bit shifting and masking; parity computation; status-flag manipulation.
Data Structures	Templated circular buffer for transmit and receive queues, with overflow detection.
OOP in C++	Peripheral base class with polymorphic Uart (and optional Spi) derived classes; Bus dispatching over base-class pointers.
Lambdas & Callbacks	Event-hook system for byte-received, buffer-overflow, and framing-error interrupts, attached as lambda functions.

8. References

- ARM Limited, PL011 UART Technical Reference Manual.
- Texas Instruments, 16550 UART Data Sheet.
- TIA/EIA-232-F, Interface Between Data Terminal Equipment and Data Circuit-Terminating Equipment (RS-232 standard).
- B. Stroustrup, The C++ Programming Language, 4th Edition.
- ISO/IEC 14882, Programming Languages - C++ (C++ Standard).
- Course lecture notes and lab material, C/C++ for Hardware Engineers module.